

Human-like Abstract Visual Reasoning via Understanding and Solving Reasoning Loop

Xinwang Chen¹ Xiuxing Li^{1,✉} Qing Li¹ Ziyue Zhuang¹ Yutong Wu¹
Ziyu Li¹ Zhuo Wang¹ Kai Li² Jianye Hao² Xia Wu¹

¹School of Computer Science and Technology, Beijing Institute of Technology

²Huawei Noah’s Ark Lab

¹{chenxinwang, xxl, liqing, wuxia}@bit.edu.cn

Abstract

*Abstract visual reasoning benchmarks such as ARC-AGI evaluate the ability to infer generalizable transformation rules from few graphical demonstrations, a capability where current deep learning models severely underperform. Mainstream LLMs achieve only 15.8% (DeepSeek-R1) and 34.5% (o3-mini-high) accuracy. The core reason lies in their static processing of task examples: unlike humans, who iteratively refine their understanding of examples while solving problems, these models lack mechanisms for dynamically aligning understanding and solving. We address this gap with the **Understanding and Solving Reasoning Loop (USRL)** framework. The architecture comprises two explicitly interacting modules: an **Understanding Module (UM)** that encodes and refines rule representations of examples, and a **Solving Module (SM)** that generates a draft solution informed by these evolving rule representations. Through recurrent interaction, the model iteratively aligns its draft solution with its understanding about task examples continuously. Furthermore, we introduce an **adaptive reasoning halting mechanism** that autonomously terminates the reasoning loop based on the consistency between the generated draft solution and the examples. With 7M parameters, our model achieves 47.2% accuracy on ARC-AGI-1.*

1. Introduction

Abstract reasoning is a hallmark of human intelligence and remains a central challenge for Artificial General Intelligence (AGI) [4, 15, 21, 30]. It involves the ability to infer underlying rules and generalize them to new situations from only a few examples. Although modern AI models excel at pattern recognition, they still struggle fundamentally with

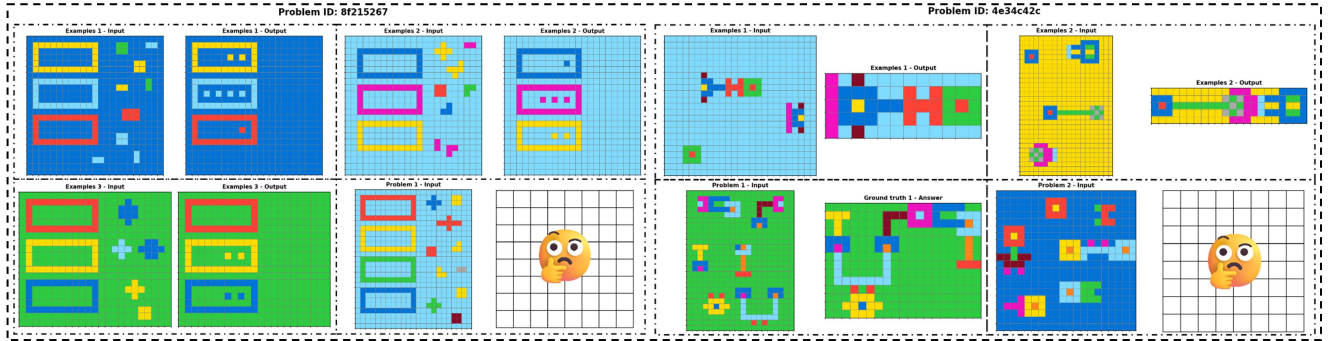
this type of reasoning [19, 21, 36]. The Abstraction and Reasoning Corpus for AGI (ARC-AGI) benchmark [6, 11] is designed to evaluate this core capability of abstract visual reasoning in artificial systems. In each task, a model must infer the underlying transformation rule from a few examples (typically 2-5) and apply it to predict the correct output for a problem input [11] as shown in Figure 1(a). Each problem represents a separate, self-contained micro-task that the model has not encountered during training, requiring it to grasp the underlying logic instead of depending on superficial statistical patterns. Therefore, ARC-AGI acts as an important benchmark for measuring progress toward general artificial intelligence.

Despite remarkable advances in other domains, state-of-the-art AI models still perform poorly on ARC-AGI. Current approaches can be broadly categorized into two groups: general-purpose large models and small task-specific architectures [5]. General-purpose models, such as large language models (LLMs), deliver only limited accuracy, yet consume enormous computational and inference-time costs. For example, DeepSeek-R1 [10] and o3-mini-high [32] reach only 15.8% and 34.5% test accuracy, respectively. On the other hand, small task-specific models such as the Tiny Recursion Model (TRM) [14] address efficiency but suffer from a rigid static design. It relies on fixed puzzle-embedding tables constructed from training task IDs, preventing dynamic example encoding and rule understanding, and thus fails to solve previously unseen tasks.

This stands in stark contrast to the human cognitive process as shown in Figure 1(b). When confronted with a reasoning task such as those in ARC-AGI, humans do not rely on a single forward computation [16, 26, 29]. Instead, they engage in an “understand–solve” reasoning loop [2, 22]: forming an initial hypothesis about the underlying rule, generating a tentative solution, assessing its consistency with the observed examples, and then refining their hypothe-

✉Corresponding author.

(a) ARC-AGI tasks



(b) “Understand and solve” reasoning loop of humans

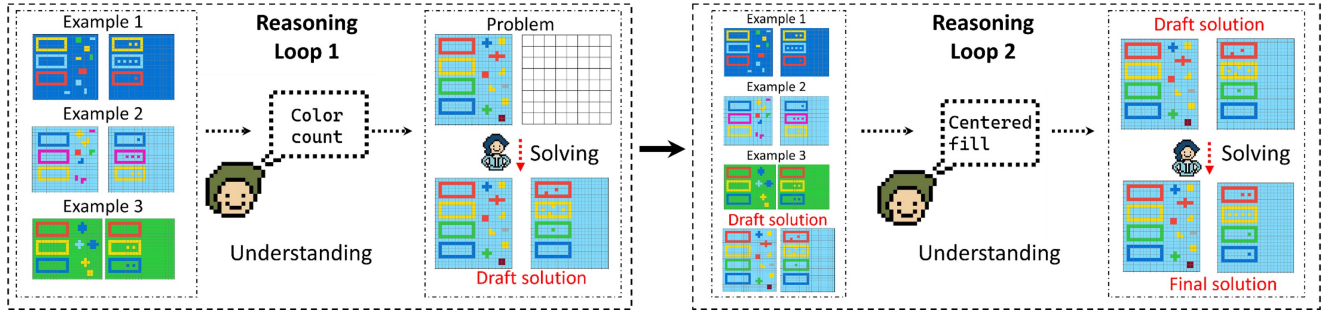


Figure 1. ARC-AGI tasks and the human reasoning loop. (a) ARC-AGI tasks: Each task consists of several input-output pairs (Examples) and a test input (Problem), requiring the model to infer underlying rules within Examples and apply rules to solve the Problem. (b) The “Understand and solve” reasoning loop of humans: Shows the cognitive process involved in solving abstract reasoning problems. Rather than deriving a solution in a single step, humans employ an iterative loop—observing examples, forming hypotheses (“Color count”), drafting solutions, identifying errors, and refining hypotheses (“Centered fill”)—to progressively approach the correct final solution.

sis accordingly [33]. Through this process loop, both understanding and problem-solving evolve jointly until the grasp of the rule and the produced solution become self-consistent [26, 28]. This iterative interplay between understanding and generation enables humans to flexibly allocate reasoning depth to task complexity—a property that is fundamentally missing from existing AI architectures.

To help models understand examples and reason over them dynamically, we draw inspiration from the reasoning loop of humans and propose the **Understanding and Solving Reasoning Loop (USRL)** framework. The core of USRL lies in the dynamic interaction between the **Understanding Module (UM)** and the **Solving Module (SM)**. The UM encodes the transformation rules from the given examples, while the SM generates a draft solution guided by the rule representation. Specifically, the UM and SM, respectively, represent the understanding of examples (rule representation) and the evolving draft solution (solution state) in the feature space. Within the reasoning loop, the rule representation produced by the UM is modulated by the current solution state and then injected back into the SM to guide its next solution state, forming a multi-step “understand–solve–re-understand” process. To ensure that

the reasoning loop terminates within a finite number of steps, we further design an **adaptive reasoning halting mechanism**, which stops the loop once the solution state becomes aligned with the rule representation. Since each module reuses its own parameters across reasoning steps, the overall parameter count remains small. Overall, this design enables small task-specific models to transcend fixed puzzle-embedding tables, dynamically understand and apply abstract rules, and self-adapt their reasoning steps to the task’s complexity.

This work makes three main contributions: (1) We introduce the **Understanding and Solving Reasoning Loop (USRL)** framework, which enables small models to perform human-like iterative reasoning by recurrently aligning task understanding and solution. (2) We design an **Understanding Module (UM)** that abstracts rule representations from examples and supports flexible reasoning loop. (3) We propose an **adaptive reasoning halting mechanism** that lets the model self-adaptively determine the reasoning steps based on internal representation consistency. (4) With 7M parameters, our model **achieves 47.2% on the ARC-AGI-1**, demonstrating the effectiveness of our neurocognitive-inspired design.

2. Related Work

ARC-AGI Abstract Visual Reasoning Tasks. ARC-AGI is a benchmark for evaluating general intelligence under conditions of extreme data scarcity, each task consists of only 3-5 input-output example pairs [4, 5]. These tasks require models to infer abstract transformation rules that generalize to new inputs, without relying on surface-level statistical correlations. Existing approaches can be broadly grouped into two paradigms: *program synthesis* (induction) [8, 23, 27], which aims to generate symbolic or programs that explain the examples; and *test-time training* (TTT) (transduction) [1, 5, 18], which directly maps new inputs to outputs by adapting a pretrained model using the few available examples. Recent research suggests that these approaches are complementary [17]: induction excels at precise and compositional logic, while transduction better handles perceptual ambiguities and noisy patterns.

Iterative Reasoning and Adaptive Computation. A core feature of human cognition and reasoning is its iterative nature, which unfolds through loops of hypothesis formation, solution drafting, and hypothesis revision [2, 22, 26]. Adaptive Computation Time (ACT) [9] operationalizes this idea in neural networks, allowing models to dynamically determine how many steps to allocate on a given input. This principle extends to the Universal Transformer [7], which uses ACT-controlled recurrence to allocate varying depth per token. Recent work shows that even small models can benefit from such iterative dynamics. TRM [14], with only 7M parameters, feeds its own latent state outputs (or prediction) back as input recurrently and achieves competitive performance (44.6% on ARC-AGI-1). This reveals that recursive refinement, rather than scale alone, can significantly improve generalization and alignment between understanding and solution generation.

Representation Learning for Task Rules. Abstract reasoning requires models to construct internal representations of implicit rules from very limited supervision. Recent strategies for such “rule learning” fall into three major categories: (1) *Task adapter*, which encode rule shifts as weight-space deltas between a pretrained and fine-tuned model ($\tau = \theta_{\text{finetuned}} - \theta_{\text{pretrained}}$), as used in TTT’s low-rank adaptation [1]; (2) *Condition injection*, where external information modulates the model’s internal computation [24, 25], often using FiLM layers [25] or AdaLN [3, 24] to conditionally transform activations; (3) *Task embedding*, where encoders [18] map few-shot examples, or embedding tables [14] map task IDs, into a vector representation used to guide downstream reasoning [31]. Our UM synthesizes (2) and (3) by learning latent rule representations that are dynamically modulated by the current solution state of SM.

3. Methods

Our methodology draws inspiration from a neurocognitive insight: human abstract reasoning is not a single-pass computation, but an iterative process that continuously refines the understanding of examples while simultaneously constructing the solution [16, 26, 29]. We operationalize this insight through a novel reasoning loop framework. This section first presents the overall framework, illustrating the dynamic interaction between its core modules (the Understanding Module and the Solving Module). Next, we describe the Understanding Module and its training procedure in detail. Finally, we explain the recurrent reasoning process and adaptive reasoning halting mechanism.

3.1. Overall Framework

Inspired by the human cognitive loop of understanding and solving, we formulate the ARC-AGI task as a dynamic recurrent framework. Previous approaches [14] typically employ fixed puzzle embeddings, which are incapable of modeling the human-like reasoning loop dynamics and fail to generalize to new puzzles. In contrast, **Understanding and Solving Reasoning Loop (USRL)** continuously generates and refines rule representations and draft solutions in Figure 2(a).

Formally, each ARC-AGI problem i is defined by m_i example pairs $E_i = \{(x_{i,k}, y_{i,k})\}_{k=1}^{m_i}$, together with a problem input x_i . Its answer is y_i . The objective is to infer the underlying transformation rule from E_i and apply it to predict y_i for the given x_i . To achieve this, our architecture integrates two explicitly interacting modules:

- (1) **Understanding Module (UM)**, denoted as U_θ , which encodes the transformation rules from the example set E_i into a problem-level **rule representation** e_i .
- (2) **Solving Module (SM)**, denoted as S_θ , implemented as a Recurrent Transformer [14] that maintains and updates a latent solution state z_i to construct the solution for the problem input x_i iteratively.

The core of our method lies in the recurrent interaction between these two modules, mirroring the iterative nature of the human reasoning loop [26]. Instead of performing a static, one-shot forward pass, the modules interact over multiple steps (indexed by t). Let $z_{i,t-1}$ denote the solution state from the step $t-1$. At the step t , the process unfolds as follows:

- (1) **Rule Understanding:** The UM U_θ processes the full example set E_i to produce an initial or “raw” rule representation $e'_i = U_\theta(E_i)$ (which can be computed once and cached).
- (2) **Rule representation modulation:** Instead of injecting e'_i into SM directly, e'_i is *modulated* by the current solution state $z_{i,t-1}$ through a learned gating g_θ . This produces the modulated rule representation $\tilde{e}_{i,t-1} = g_\theta(z_{i,t-1}, e'_i)$, which enables the model to selectively

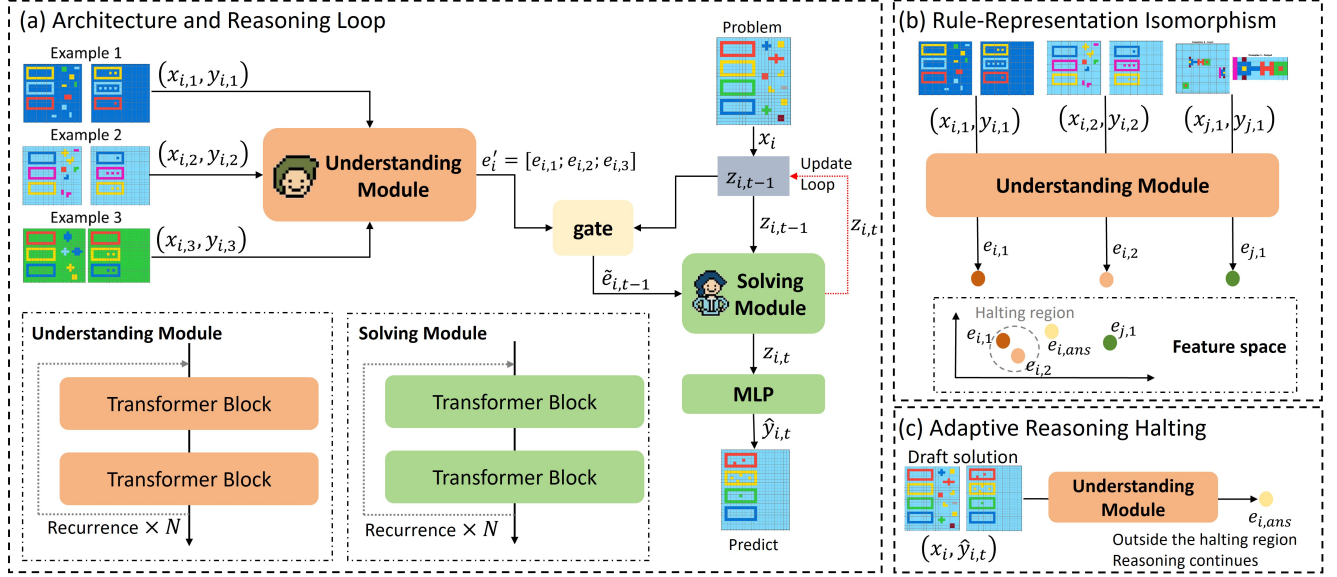


Figure 2. Architecture designed based on the human reasoning loop. (a) **Architecture and Reasoning Loop**: The model comprises UM and SM, both utilizing recurrent Transformer blocks. Through multiple interactions between UM and SM, the model continuously updates the rule representation $\tilde{e}_{i,t-1}$ and the solution state $z_{i,t}$ to progressively optimize the prediction $\hat{y}_{i,t}$. (b) **Rule-Embedding Isomorphism**: Illustrates how the UM maps example pairs into a feature space. Examples sharing the same transformation rule ($e_{i,1}, e_{i,2}$ from task i) are clustered closely together, while those with different rules ($e_{j,1}$) are pushed apart, defining a specific “Halting region” for task i . (c) **Adaptive Reasoning Halting**: The generated draft solution from SM is fed back into UM. If its rule representation $e_{i,ans}$ falls within the halting region, the reasoning loop terminates to output the final result; otherwise, it proceeds to the next reasoning loop.

emphasize or re-weight elements of the raw rule representation (e'_i) according to its current “solution draft” ($z_{i,t-1}$).

- (3) **Solution Update**: The SM S_θ integrates the problem input x_i , the previous state $z_{i,t-1}$, and the modulated rule representation $\tilde{e}_{i,t-1}$ to update its solution state: $z_{i,t} = S_\theta(x_i, z_{i,t-1}, \tilde{e}_{i,t-1})$.
- (4) **Solution Output**: The updated state $z_{i,t}$ is decoded via a linear projection f_θ to yield a draft solution $\hat{y}_{i,t} = f_\theta(z_{i,t})$.

The complete inference procedure is summarized in Table 1. This reasoning loop $(E_i, x_i) \rightarrow (e'_i, z_{i,t-1}) \rightarrow \tilde{e}_{i,t-1} \rightarrow z_{i,t} \rightarrow \hat{y}_{i,t}$ progressively refines the latent solution state until the model’s understanding and generated solution reach self-consistency, as detailed in Section 3.3.

3.2. Understanding Module

The Understanding Module (UM) U_θ forms the core of our framework, responsible for understanding and extracting transformation rules from the provided examples E_i .

3.2.1. Architecture

Owing to the limited size of the ARC-AGI dataset (approximately 1000 training tasks), the UM adopts a structurally simple, parameter-sharing design consisting of shared Transformer blocks. This mitigates the risk of overfitting.

To enhance expressivity under limited parameters, the output of UM is fed back as input in multiple internal recurrences, enabling the iterative refinement of raw rule representations.

The procedure for encoding the example set $E_i = \{(x_{i,k}, y_{i,k})\}$ is as follows:

- (1) Each example input $x_{i,k}$ and output $y_{i,k}$ is tokenized and embedded as $h_{i,k}^x, h_{i,k}^y \in \mathbb{R}^{L \times D}$, where L is the sequence length of the tokenized grid and D is the embedding dimension.
- (2) A set of L_q **query tokens** $\mathbf{Q}_k \in \mathbb{R}^{L_q \times D}$ (initialized to zero) is concatenated with the projected example embeddings to form the combined sequence $\mathbf{H}_{i,k} = [\mathbf{Q}_k \parallel \text{Linear}([h_{i,k}^x \parallel h_{i,k}^y])]$, where $\parallel$ denotes concatenation and the linear layer maps the concatenated input-output embeddings back to dimension D .
- (3) The sequence $\mathbf{H}_{i,k}$ is passed through UM with multiple internal recurrences, enabling it to capture the complex transformations within examples.
- (4) The final hidden states corresponding to the query tokens $\mathbf{Q}_k$ are extracted as the example-specific rule representation $e_{i,k} \in \mathbb{R}^{L_q \times D}$.
- (5) These representations are collected across all examples to form the problem-level raw rule representation set $e'_i = \{e_{i,k}\}_{k=1}^{m_i}$, which is then passed to the learnable gating mechanism g_θ described in Section 3.1.

Table 1. USRL Inference procedure.

Algorithm 1: USRL Inference Process

Require: Example set $E_i = \{(x_{i,k}, y_{i,k})\}$, problem input x_i
Require: Max reasoning steps $T_{\max}$
Require: Modules: U_θ (UM), S_θ (SM), g_θ (Gate), f_θ (Decoder)
Ensure: Predicted test output $\hat{y}_i$

```

1:  $\hat{y}_{i,0}, z_{i,0} \leftarrow \text{InitializeState}()$  #Init solution state
2: # 1. Understand (§3.2)
3:  $e_{i,\text{ans}} \leftarrow U_\theta(\{(x_i, \hat{y}_{i,0})\})$  #Answer representation
4:  $e'_i \leftarrow U_\theta(E_i)$  #Raw rule representations
5:  $C_{\text{intra}} \leftarrow \text{AvgSim}(e'_i)$  #Halting threshold
6: for  $t = 1$  to  $T_{\max}$  do
7:   # 2. Modulate (§3.1)
8:    $\tilde{e}_{i,t-1} \leftarrow g_\theta(z_{i,t-1}, e'_i)$  #Modulate rules
9:   # 3. Solve (§3.1)
10:   $z_{i,t} \leftarrow S_\theta(x_i, z_{i,t-1}, \tilde{e}_{i,t-1})$  #Solution state
11:  # 4. Solution Output (§3.1)
12:   $\hat{y}_{i,t} \leftarrow f_\theta(z_{i,t})$  #Decode draft solution
13:  # 5. Adaptive Halting (§3.3.2)
14:   $e_{i,\text{ans}} \leftarrow U_\theta(\{(x_i, \hat{y}_{i,t})\})$ 
15:   $C_{\text{ans}} \leftarrow \text{AvgSim}(e_{i,\text{ans}}, e'_i)$  #Similarity
16:  if  $C_{\text{ans}} > C_{\text{intra}}$  then
17:    return  $\hat{y}_{i,t}$  #Halt: solution is consistent
18:  end if
19: end for
20: return  $\hat{y}_{i,T_{\max}}$  #Return last solution

```

3.2.2. Training based on Representation Isomorphism

We hypothesize that example pairs $(x_{i,k}, y_{i,k})$ and $(x_{i,j}, y_{i,j})$ from the same problem i share an identical underlying transformation rule. Therefore, their corresponding example-specific representations $e_{i,k}$ and $e_{i,j}$ should be isomorphic, meaning they are mapped closely together in the representation space as shown in Figure 2(b).

Empirically, we observe that this clustering phenomenon indeed emerges spontaneously during training. When training the model only using the primary solution loss $\mathcal{L}_{\text{CE}}(y, \hat{y})$, the similarities among example-specific representations $e_{i,k}$ from the same problem i naturally increase as training progresses. This indicates that the end-to-end interaction between UM and SM inherently encourages the model to capture and reinforce shared transformation rules across examples.

To strengthen this emergent property, we introduce an auxiliary contrastive loss $\mathcal{L}_{\text{contrast}}$, formulated as a supervised InfoNCE objective. Let V denote the set of all example-specific representations $e_{i,k}$ in a batch. For each representation $e_{i,k} \in V$, let $P(e_{i,k}) \subset V$ denote the set of positive samples, comprising representations extracted from the same problem i , excluding $e_{i,k}$ itself. The set of all other samples (positives and negatives) is defined as $A(e_{i,k}) = V \setminus \{e_{i,k}\}$. The contrastive loss for embedding

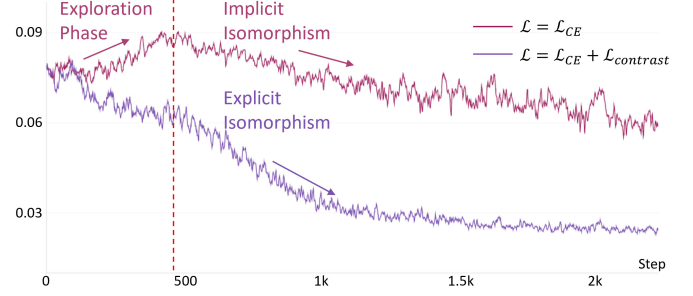


Figure 3. Comparison of Rule Representation Isomorphism Evolution. The trajectories of contrastive loss (lower is better) under two training paradigms. Red Line (Baseline, $\mathcal{L}_{\text{CE}}$): Exhibits an initial increase (Exploration Phase) followed by a slow, spontaneous decline (Implicit Isomorphism), indicating weak rule alignment. Purple Line (Ours, $\mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{contrast}}$): With the auxiliary loss, the model bypasses the exploration instability and rapidly achieves Explicit Isomorphism, converging to a significantly lower loss value. This demonstrates that explicit contrastive constraints force the model to construct a more discriminative feature space.

$e_{i,k}$ is defined as:

$$\mathcal{L}_{e_{i,k}} = -\frac{1}{N} \sum_{p \in P(e_{i,k})} \log \frac{\exp(s(e_{i,k}, p)/\tau)}{\sum_{a \in A(e_{i,k})} \exp(s(e_{i,k}, a)/\tau)}$$

where $N = |P(e_{i,k})|$, $s(\cdot, \cdot)$ denotes cosine similarity, and τ is a temperature hyperparameter. The total contrastive loss is the average over all embeddings in the batch:

$$\mathcal{L}_{\text{contrast}} = \frac{1}{|V|} \sum_{e_{i,k} \in V} \mathcal{L}_{e_{i,k}}$$

This objective forces the UM to map similar rules to nearby representations, while pushing representations of different rules apart. The final training objective combines the primary solution loss with the auxiliary contrastive term:

$$\mathcal{L} = \mathcal{L}_{\text{CE}}(y, \hat{y}) + \lambda_c \mathcal{L}_{\text{contrast}}$$

where λ_c controls the balance between the two objectives. As shown in Figure 3, the addition of this auxiliary loss significantly accelerates the convergence of rule isomorphism.

3.3. Understanding and Solving Reasoning Loop

Having established how UM learns rule-consistent representations, we now describe how these representations interact with SM in a reasoning step. This section details the dynamics of the outer recurrent reasoning and our adaptive reasoning halting mechanism.

3.3.1. Training for Reasoning Loop

The ‘‘Understand–Solve–Re-understand–Solve’’ loop described in Section 3.1 does not operate with a fixed number of reasoning steps. Instead, the reasoning process is

governed by an Adaptive Computation Time (ACT) mechanism, which enables the model to allocate computation dynamically according to task complexity and solution state. Intuitively, the model “thinks longer” for more complex puzzles and halts earlier for simpler ones, reflecting a variable-depth reasoning process analogous to human cognition.

To encourage the model to learn multi-step reasoning behavior during training, we explicitly optimize it at each intermediate reasoning step rather than only at the final output. At each reasoning step, the model produces a draft solution $\hat{y}_{i,t}$, for which the loss $\mathcal{L}$ is immediately computed and backpropagated. This step-wise optimization allows UM and SM to progressively refine their parameters as reasoning unfolds.

To avoid all training samples within a batch being processed at identical reasoning depths, we introduce a **stochastic halting**. It probabilistically terminates reasoning of some samples before reaching the maximum reasoning step and replaces them with new samples. This dynamic resampling strategy maintains a balanced distribution of reasoning depths, allowing the model to experience diverse reasoning trajectories during training and mitigating overfitting to a fixed reasoning length.

3.3.2. Adaptive Reasoning Halting

A key challenge in ACT is determining when to stop the reasoning loop during inference. The Rule Representation Isomorphism observed in Section 3.2 suggests that examples $(x_{i,k}, y_{i,k}) \in E_i$ within the same problem naturally cluster in the representation space, indicating a shared underlying transformation rule. Building upon this property, the reasoning loop should terminate once the model’s understanding (captured by the rule representations) and its draft solution align closely within the representation space as shown in Figure 2(c).

This mechanism operates as follows:

- (1) At the reasoning step t , we obtain the example-specific representations $e'_i = \{e_{i,k}\}$ and the current draft solution $\hat{y}_{i,t}$.
- (2) The pair $(x_i, \hat{y}_{i,t})$ is treated as a *hypothetical new example* and passed through the UM U_θ to produce an **answer representation**: $e_{i,\text{ans}} = U_\theta(\{(x_i, \hat{y}_{i,t})\})$.
- (3) Two cosine-similarity metrics are then computed:
 - **Intra-Example Consistency (C_{intra}):** The average pairwise similarity among the representations of known examples, representing the internal coherence of the inferred rule space:

$$C_{\text{intra}} = \text{avg}_{j \neq k}(\text{sim}(e_{i,j}, e_{i,k}))$$

- **Answer-Example Consistency (C_{ans}):** The average similarity between the hypothetical answer representation and all example representations, assessing how

well the current solution aligns with the inferred transformation rule:

$$C_{\text{ans}} = \text{avg}_k(\text{sim}(e_{i,\text{ans}}, e_{i,k}))$$

- (4) **Halting Condition:** The reasoning halts once the draft answer is as consistent with the examples as the examples are with each other (Halt if $C_{\text{ans}} > C_{\text{intra}}$).

This self-consistency criterion allows the model to automatically adjust its reasoning depth based on the convergence of its internal representations. By halting when the generated solution and the inferred rule representations coincide in the semantic space, the model learns to balance computational cost and reasoning accuracy without explicit supervision.

4. Experiments

In this section, we evaluate the effectiveness of the USRL framework on the ARC-AGI benchmark. Our experiments address three key areas: (1) A comparison of USRL’s accuracy against LLMs and task-specific models. (2) An analysis of the contribution of each component to the overall performance.

4.1. Experimental Setup

Dataset. We evaluate our model on the ARC-AGI-1 benchmark [6], which contains 400 training tasks and 400 evaluation tasks. We also adopt the data augmentation scheme from TRM [14], applying augmentations such as color permutations and geometric transformations, expanding the training data to 876k tasks. Following standard protocols, we train on this augmented set and report performance on unseen evaluation set. Please see Supplementary Materials for more details.

Implementation Details. UM and SM are implemented as Recurrent Transformers, each comprising two blocks. The model has a total of 7M parameters, with an embedding dimension of $D = 368$ and 8 attention heads. The maximum number of reasoning steps T_{max} is 16. During training, the temperature τ for the contrastive loss (Section 3.2) is set to 0.07, and the balancing coefficient λ_c is set to 0.5. Training is performed on 8 NVIDIA L40s GPUs and 16 NVIDIA V100 GPUs. For inference, we employ the proposed Adaptive Reasoning Halting mechanism from Section 3.3. The model dynamically decides when to terminate the reasoning loop based on the described consistency metrics. If the halting condition is not met, the loop is forcibly terminated at T_{max} . We generate 2 answers for each problem (pass@2). Please see Supplementary Materials for more training details.

4.2. Performance Evaluation

As shown in Table 2, we compare USRL against two categories of baselines: (1) general-purpose LLMs, including

o3-mini-high, Gemini 2.5 Pro 32K, and DeepSeek-R1 [10]; (2) method combining fine-tuned Qwen2.5-0.5B and TTT such as Omni-ARC [5]; and (3) specialized architectures such as HRM [32] and TRM [14].

On the ARC-AGI-1 evaluation set, USRL achieves an accuracy of 47.2%, marking the highest performance among all evaluated models. Compared with other task-specific systems, USRL outperforms TRM (44.6%), despite both models sharing the same 7M parameter count. It also surpasses the larger HRM (27M), which achieves 40.3%. USRL also significantly outperforms the general-purpose LLMs included in the comparison. Specifically, it achieves higher accuracy than Gemini 2.5 Pro 32K (37.0%), o3-mini-high (34.5%), and DeepSeek-R1 (15.8%). These results demonstrate the competitiveness and efficacy of the USRL framework in abstract visual reasoning.

Table 2. Performance comparison on the ARC-AGI-1 evaluation set (2 tries).

Model	Params.	Accuracy (%)
<i>LLMs</i>		
DeepSeek-R1	~671B	15.8
o3-mini-high	-	34.5
Gemini 2.5 Pro 32K	-	37.0
Omni-ARC	0.5B	40.0
<i>Task-specific Models</i>		
HRM	27M	40.3
TRM	7M	44.6
USRL (Ours)	7M	47.2

4.3. Ablation Studies

To understand the source of our performance gains, we conduct a comprehensive ablation study focusing on architectural components and the reasoning loop design. Additional experiments can be seen in Supplementary Materials.

4.3.1. Component Analysis

Table 3 presents the contribution of each component in USRL architecture. The baseline model, which contains only SM (3.3M parameters), achieves an accuracy of 36.5%.

The first major improvement comes from adding UM without gate, which doubles the parameter count to 6.7M and increases accuracy by +4.3% to 40.8%. This demonstrates the importance of explicitly decoupling task understanding from solution generation. Introducing the gating mechanism (Section 3.1) brings an additional +2.4% gain (to 43.2%), supporting our hypothesis that the rule representation should be dynamically modulated based on the evolving solution state.

Next, applying the Contrastive Loss ($\mathcal{L}_{contrast}$) during training yields another +3.2% gain (to 46.4%). This

demonstrates that enforcing rule-representation isomorphism (Section 3.2) in the feature space successfully guides the model to learn more robust and generalizable task representations. The Stochastic Halting training strategy (Section 3.3) adds a final +0.8%, propelling the model to its peak accuracy of 47.2% (when measured using a fixed inference step).

Finally, we evaluate our Adaptive Reasoning Halting mechanism (Section 3.3) for inference. This method achieves 47.0%, a negligible 0.2% drop from the peak score. This validates that our consistency-based halting criterion determines the required reasoning steps, achieving near-optimal accuracy while providing computational flexibility.

Table 3. Ablation study of key components.

Configuration	Params.	Acc. (%)	Δ
Base (Solving Module only)	3.3M	36.5	-
+ Understanding Module (w/o Gate)	6.7M	40.8	+4.3
+ Understanding Module (w/ Gate)	7.0M	43.2	+2.4
+ Contrastive Loss ($\mathcal{L}_{contrast}$)	7.0M	46.4	+3.2
+ Stochastic Halting (training)	7.0M	47.2	+0.8
+ Adaptive Reasoning Halting (inference)	7.0M	47.0	-0.2

4.3.2. Analysis of Reasoning Steps T_{max} during Inference

We now investigate the impact of the maximum reasoning steps, T_{max} , during inference on the model’s performance and its adaptive behavior. Note that the model was trained using a fixed $T_{max} = 16$ (with stochastic halting). We vary T_{max} from 1 to 20 and report accuracy and the average number of steps (T_{Mean}) utilized by the Adaptive Reasoning Halting mechanism. The results are summarized in Table 4.

The results clearly demonstrate that sufficient reasoning depth is critical. When T_{max} is constrained (8 or less), the model is forced to terminate prematurely, resulting in poor accuracy (only 11.4% at $T_{max} = 8$). Performance increases substantially as the computational budget is expanded, rising to 25.1% at $T_{max} = 12$ and peaking at 47.0% when $T_{max} = 16$.

Increasing the budget further to $T_{max} = 20$ yields a nearly identical accuracy of 47.1%. This confirms that the model, having been trained with 16 steps, does not derive significant benefit from an expanded inference budget. This finding also indicates that $T_{max} = 16$ represents a sufficient computational budget for the model to reach its peak performance.

Even at the $T_{max} = 16$ setting, the mean number of steps used (T_{Mean}) is only 12.4 (and 13.2 for $T_{max} = 20$). This demonstrates the effectiveness and efficiency of our adaptive halting mechanism: it achieves the model’s peak accuracy without always running to the maximum step count,

allocating computation adaptively based on task complexity.

Table 4. Impact of maximum reasoning steps (T_{max}) at inference on the mean steps used (T_{Mean}) by the adaptive halting mechanism and the final accuracy (2 tries).

Max. Steps (T_{max})	Mean Steps (T_{Mean})	Accuracy (%)
1	1.0	0.5
2	1.9	2.1
4	3.7	3.7
8	7.5	11.4
12	9.3	25.1
16	12.4	47.0
20	13.2	47.1

4.4. Qualitative Analysis

To provide insight into the USRL’s internal dynamics, Figure 4 visualizes the reasoning loop on a sample task. The task (top panel) requires model identifying 3×3 squares and changing its color.

The reasoning trajectory (bottom panel) illustrates a process of iterative refinement. The model begins at Loop 0 and progressively transforms its draft solution (Loops 1, 4, 8, 12) as UM and SM interact.

As shown, the model converges to the correct solution at Loop 16. The key observation is the behavior of the adaptive halting mechanism: the loop correctly terminates (faded steps 17, 18) once the internal solution state becomes self-consistent with the rule representation. This confirms the mechanism can prevent unnecessary computation and reasoning steps.

5. Conclusions and Limitations

In this work, we introduced the Understanding and Solving Reasoning Loop (USRL), a neurocognitive-inspired framework for abstract visual reasoning. By explicitly separating the reasoning process into an Understanding Module (UM) and a Solving Module (SM) and enabling recurrent interaction between them, the model iteratively refines its understanding of latent transformation rules from a few examples. With 7M parameters, USRL achieves 47.2% accuracy on the ARC-AGI-1 evaluation set, surpassing LLMs and prior specialized models. The main contribution of USRL lies in the architectural design. Unlike static, feed-forward approaches, the “understand-solve” loop of USRL provides a structural mechanism that enables the model to infer transformation rules from examples and apply them to solve novel problems. Our ablation studies show that this capability emerges from the interplay between UM and SM rather than from any single module alone. These findings suggest that for challenging reasoning tasks such as ARC, a

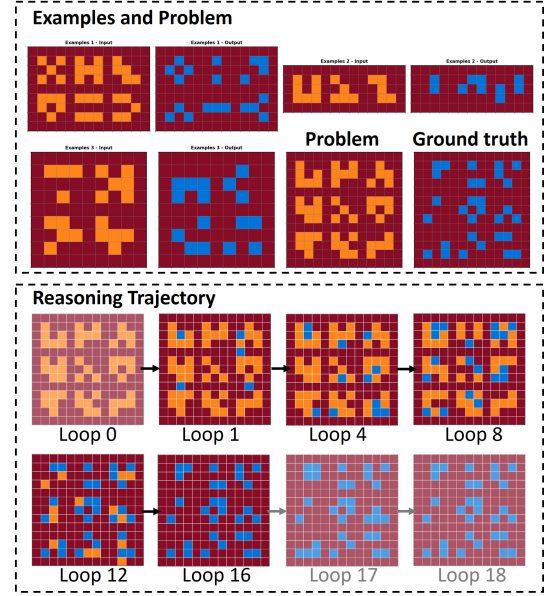


Figure 4. Qualitative reasoning trajectory of the USRL framework on an ARC-AGI task. The Top panel (Examples and Problem) displays the task context, including three input-output examples, the test input (Problem), and the corresponding Ground Truth. The Bottom panel (Reasoning Trajectory) visualizes our model’s iterative reasoning process. The model begins with the initial problem state (Loop 0) and progressively refines its solution through the USRL loop. At Loop 16, the model’s solution state converges and perfectly matches the Ground Truth. The subsequent faded-out steps (Loops 17, 18) illustrate the adaptive reasoning halting mechanism activating after convergence is detected, terminating the loop.

compact yet carefully designed architecture can outperform models that rely on scale.

ARC-AGI abstracts real-world imitation and reasoning problems by requiring models to infer latent rules from a few examples and generalize. USRL targets tasks like ARC-AGI, but isn’t suited for fixed-rule problems (like Sudoku). It depends on example-based rule induction and is not intended for tasks expressed in formats that do not provide demonstration pairs. Moreover, the 47.2% accuracy remains below human performance, indicating that many abstract reasoning patterns remain difficult for USRL. Future directions include extending this reasoning loop to other domains [12, 13, 34, 35] that follow rule-induction or demonstration-based paradigms.

Acknowledgement

This work was supported in part by the National Natural Science Foundation of China under Grants 62325601, 62236001, and 92470125.

References

- [1] Ekin Akyürek, Mehul Damani, Linlu Qiu, Han Guo, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for abstract reasoning. *arXiv e-prints*, pages arXiv–2411, 2024. 3
- [2] Jesús Antonio Cerón-Molina, Myrian-Elena Vergara-Morales, and Camilo Andrés Rodríguez-Nieto. Proposal for substantive theory: A connection between mathematics learning and programming for children. *Eurasia Journal of Mathematics, Science and Technology Education*, 21(9): em2696, 2025. 1, 3
- [3] Xinwang Chen, Ning Liu, Yichen Zhu, Feifei Feng, and Jian Tang. Edt: An efficient diffusion transformer framework inspired by human-like sketching. *Advances in Neural Information Processing Systems*, 37:134075–134106, 2024. 3
- [4] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019. 1, 3
- [5] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. Arc prize 2024: Technical report. *arXiv preprint arXiv:2412.04604*, 2024. 1, 3, 7
- [6] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. Arc-agi-2: A new challenge for frontier ai reasoning systems. *arXiv preprint arXiv:2505.11831*, 2025. 1, 6
- [7] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. *arXiv preprint arXiv:1807.03819*, 2018. 3
- [8] Daniel Franzen, Jan Disselhoff, and David Hartmann. The llm architect: Solving arc-agi is a matter of perspective, 2024. 3
- [9] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016. 3
- [10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. 1, 7
- [11] Michael Hodel. Addressing the abstraction and reasoning corpus via procedural example generation. *arXiv preprint arXiv:2404.07353*, 2024. 1
- [12] Stephen James, Zicong Ma, David Rovick Arrojo, and Andrew J Davison. Rlbench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 5(2):3019–3026, 2020. 8
- [13] Tianyuan Jia, Ziyu Li, Qing Li, Xiuxing Li, and Xia Wu. Brainymp: enhancing motion planning using graph neural network inspired by brain spatial relational memory. *IEEE Transactions on Intelligent Transportation Systems*, 2025. 8
- [14] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025. 1, 3, 6, 7
- [15] CRS Kumar. Towards artificial general intelligence: Enhancing llms capability for abstraction and reasoning. 2024. 1
- [16] Pat Langley, Lorenzo Magnani, Christian Schunn, and Paul Thagard. An extended theory of human problem solving. In *Proceedings of the annual meeting of the cognitive science society*, 2005. 1, 3
- [17] Wen-Ding Li, Keya Hu, Carter Larsen, Yuqing Wu, Simon Alford, Caleb Woo, Spencer M Dunn, Hao Tang, Michelangelo Naim, Dat Nguyen, et al. Combining induction and transduction for abstract reasoning. *arXiv preprint arXiv:2411.02272*, 2024. 3
- [18] Matthew V Macfarlane and Clément Bonnet. Searching latent program spaces. *arXiv preprint arXiv:2411.08706*, 2024. 3
- [19] Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models. *arXiv preprint arXiv:2410.05229*, 2024. 1
- [20] Arseny Moskvichev, Victor Vikram Odouard, and Melanie Mitchell. The conceptarc benchmark: Evaluating understanding and generalization in the arc domain. *arXiv preprint arXiv:2305.07141*, 2023. 1
- [21] Alhassan Mumuni and Fuseini Mumuni. Large language models for artificial general intelligence (agi): A survey of foundational principles and approaches. *arXiv preprint arXiv:2501.03151*, 2025. 1
- [22] Stellan Ohlsson. The problems with problem solving: Reflections on the rise, current status, and possible future of a cognitive research paradigm. *The Journal of problem solving*, 5(1):7, 2012. 1, 3
- [23] Simon Ouellette. Towards efficient neurally-guided program induction for arc-agi. *arXiv preprint arXiv:2411.17708*, 2024. 3
- [24] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023. 3
- [25] Ethan Perez, Florian Strub, Harm De Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI conference on artificial intelligence*, 2018. 3
- [26] Giovanni Pezzulo, Thomas Parr, and Karl Friston. The evolution of brain architectures for predictive coding and active inference. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 377(1844), 2022. 1, 2, 3
- [27] Julien Pourcel, Cédric Colas, and Pierre-Yves Oudeyer. Self-improving language models for evolutionary program synthesis: A case study on arc-agi. *arXiv preprint arXiv:2507.14172*, 2025. 3
- [28] Giacomo Rizzolatti and Laila Craighero. The mirror-neuron system. *Annu. Rev. Neurosci.*, 27(1):169–192, 2004. 2
- [29] Herbert A Simon and Allen Newell. Human problem solving: The state of the theory in 1970. *American psychologist*, 26(2):145, 1971. 1, 3
- [30] James WA Strachan, Dalila Albergo, Giulia Borghini, Oriana Pansardi, Eugenio Scaliti, Saurabh Gupta, Krati Saxena, Alessandro Rufo, Stefano Panzeri, Guido Manzi, et al. Testing theory of mind in large language models and humans. *Nature Human Behaviour*, 8(7):1285–1295, 2024. 1
- [31] Kristi Topollai and Anna Choromanska. Task-level contrastiveness for cross-domain few-shot learning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 6489–6499, 2025. 3

- [32] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025. [1](#), [7](#)
- [33] Yingxu Wang and Vincent Chiew. On the cognitive process of human problem solving. *Cognitive systems research*, 11 (1):81–92, 2010. [2](#)
- [34] Yinhuai Wang, Qihan Zhao, Yuen Fui Lau, Runyi Yu, Hok Wai Tsui, Qifeng Chen, Jingbo Wang, Jiangmiao Pang, and Ping Tan. Humanx: Toward agile and generalizable humanoid interaction skills from human videos. *arXiv preprint arXiv:2602.02473*, 2026. [8](#)
- [35] Kun Wu, Ning Liu, Zhen Zhao, Di Qiu, Jinming Li, Zhengping Che, Zhiyuan Xu, and Jian Tang. Learning from imperfect demonstrations with self-supervision for robotic manipulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 16899–16906. IEEE, 2025. [8](#)
- [36] Zhaofeng Wu, Linlu Qiu, Alexis Ross, Ekin Akyürek, Boyuan Chen, Bailin Wang, Najoung Kim, Jacob Andreas, and Yoon Kim. Reasoning or reciting? exploring the capabilities and limitations of language models through counterfactual tasks. *Association for Computational Linguistics*, 2024. [1](#)